

Slovenská technická univerzita v Bratislave
Fakulta elektrotechniky a informatiky

Technická dokumentácia

Zdieľané supervízne riadenie autonómnej skupiny
dronov vo virtuálnej realite

Typ dokumentu: technická dokumentácia a používateľská príručka
Rozsah spracovania: **DroneApp**, **ros2_ws**, **doc**
Cielové prostredie: ROS2, MAVROS, GStreamer, Gazebo/ArduPilot SITL, Pico Neo 3
Rozhranie operátora: desktopová aplikácia a VR zobrazenie

29. mája 2026

Obsah

1	Úvod	2
2	Základná koncepcia systému	2
3	Architektúra systému	3
3.1	Harvérové komponenty	4
3.2	Desktopová aplikácia DroneApp	4
3.3	Pracovný priestor <code>ros2_ws</code>	5
4	Komunikácia medzi komponentmi	6
4.1	Dron a backendová aplikácia	6
4.2	Backendová a frontendová aplikácia	6
4.3	Frontendová aplikácia a VR headset	7
5	Misie a pracovný tok	8
6	Handover mechanizmus	8
7	Video streamovanie	9
8	Testovacie spustenie	9
9	Záver	10

1 Úvod

Tento dokument popisuje systém zdieľaného supervízneho riadenia autonómnej skupiny dronov vo virtuálnej realite.

Cieľom dokumentu je zrozumiteľne popísať architektúru riešenia, úlohu jednotlivých softvérových a hardvérových komponentov, komunikačné rozhrania, princíp handover mechanizmu medzi autonómnym a manuálnym režimom.

Je určený pre developerov, ktorý budú systém rozvíjať, nie pre užívateľov, pre nich je určená samostatná používateľská príručka.

2 Základná koncepcia systému

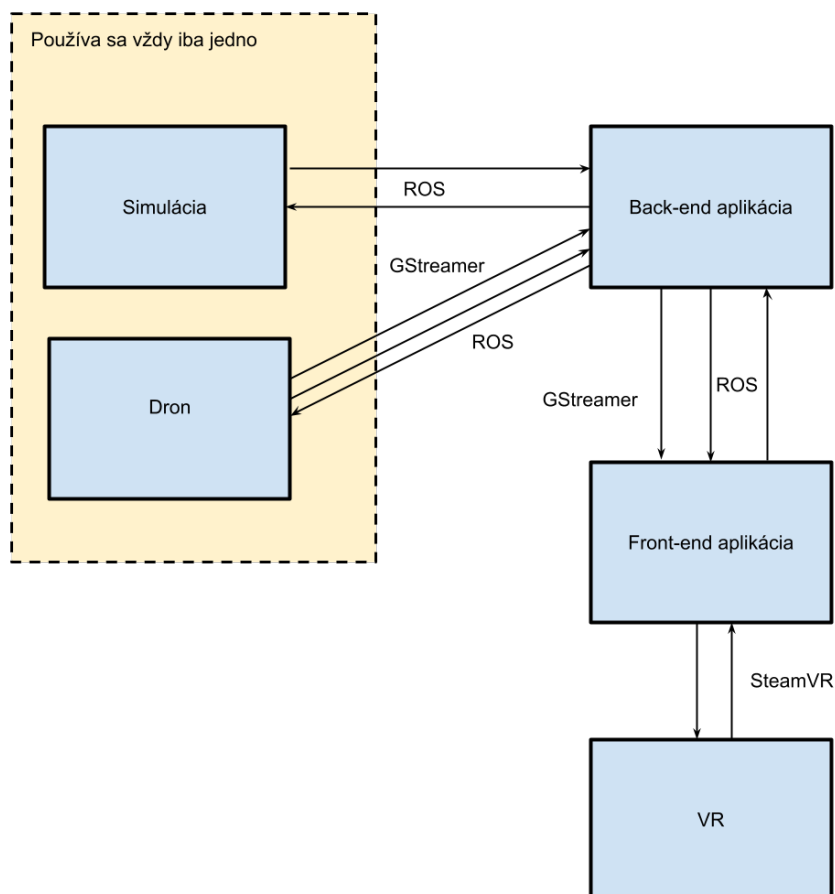
Systém funguje ako ovládacie rozhranie nad skupinou dronou. Zabezpečuje ich autonómne riadenie po preddefinovaných trasách, plynulý prechod do a z manuálneho režimu, ovládanie pomocou VR controllerov a stream videa z drona až do VR headsetu.

Všetky tieto funkcie zabezpečuje skrze spoločnú sieť s minimálnou latenciou.

Systém je rozdelený do viacerých komponentov, frontendovej používateľskej aplikácie, ktorá beží na počítači operátora a operátor je schopný pomocou nej sledovať a ovládať jednotlivé drony skrze backendovú aplikáciu. Backendová aplikácia je linuxový ROS2 server, ktorý priamo komunikuje s dronmi, prípadne simuláciou. Je schopný fungovať samostatne, kde zabezpečuje automatizované misie a bezpečný pohyb dronov. Po pripojení front end aplikácie vysiela údaje o dronoch, prijíma riadiace topiky a v prípade simulácie takisto konvertuje video z ROS2 topiku na GStreamer stream.

3 Architektúra systému

Architektúra systému je znázornená na obrázku 1:



Obr. 1: Architektúra systému

Systém sa skladá z 2 aplikácií, ktoré zabezpečujú obojsmernú komunikáciu medzi dronmi a virtuálnou realitou.

Prvou je backendová aplikácia bežiacia na ROS2, ktorá zabezpečuje komunikáciu s dronmi a automatické riadenie dronov po preddefinovaných trajektoriách. Takiež zabezpečuje bezpečnosť pri riadení dronov, vyhnutie sa kolíziám a pri simulačnom prostredí aj konverziu videa do GStreameru (reálny dron streamuje priamo cez GStreamer), ktorý následne posiela video do frontendovej aplikácie.

Frontendová aplikácia využíva sa pripája na backendovú aplikáciu pomocou ROS2 bridge cez WebSoket, čo je technológia, ktorá umožňuje prepájať ROS a ne-ROS aplikácie a GStreameru a zabezpečuje zobrazenie dronov a možnosť ovládania dronov. Na zobrazenie vo virtuálnej realite používa SteamVR a OpenVR, ktoré zabezpečujú prenos obsahu z obrazovky počítača do VR headsetu a umožňujú spracovať užívateľské vstupy z VR ovládačov.

3.1 Harvérové komponenty

Systém je navrhnutý tak, aby bol podľa kompability rozdelený do viacerých harvérových komponentov.

Z dôvodu lepšej optimalizácie ROS2 pre ubuntu linux sa na beh backendovej aplikácie používa Linux počítať s Ubuntu 22.04.

Na beh frontendovej aplikácie používate Windows počítač, pretože StreamVR je menej stabilný na Linux prostredí.

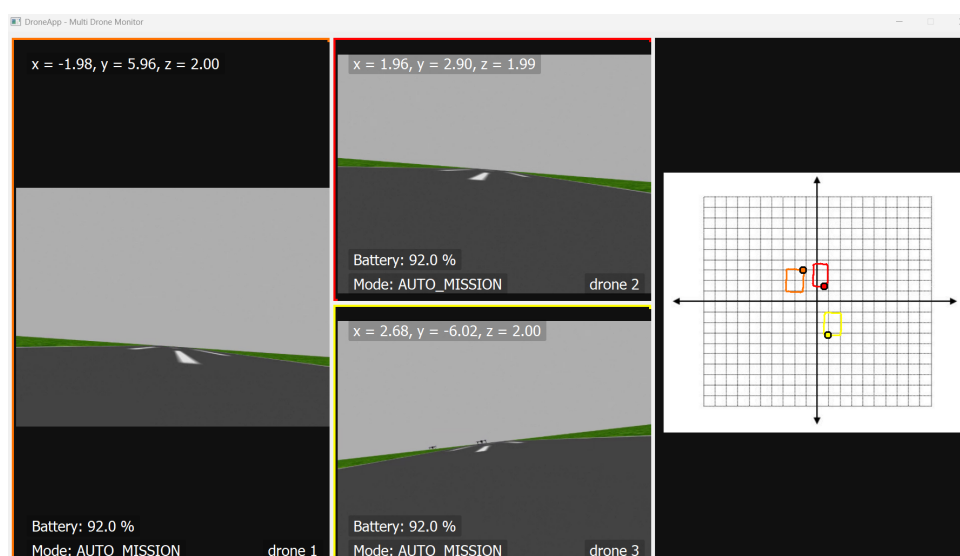
Ako ďalšie harvérové komponenty využívame Airvolute EduDrone a Pico Neo 3 VR headset s ovládačmi.

3.2 Desktopová aplikácia DroneApp

Adresár **DroneApp** obsahuje používateľskú aplikáciu implementovanú v jazyku Python s využitím frameworku PyQt5. Aplikácia slúži na monitorovanie viacerých dronov v reálnom čase a zároveň umožňuje operátorovi prevziať manuálne riadenie vybraného drona.

Grafické rozhranie pozostáva z viacerých panelov, pričom každý panel reprezentuje jeden dron. Panel zobrazuje video stream z kamery drona spolu s telemetrickými údajmi, ako sú súradnice (optitrack/GPS), stav batérie, aktuálny letový režim. Operátor môže medzi panelmi prepínať a zvoliť aktívny dron, na ktorý sa následne aplikujú príkazy manuálneho riadenia.

Aplikácia zároveň zabezpečuje príjem video streamov prostredníctvom GStreamer pipeline a komunikáciu s ROS2 prostredím cez knižnicu **roslibpy**. Táto komunikačná vrstva slúži na odber telemetrických údajov a odosielanie riadiacich požiadaviek medzi používateľským rozhraním a autonómnym systémom. Súčasťou aplikácie je aj modul mapujúci vstupy z klávesnice na rýchlostné príkazy, ktoré umožňujú operátorovi plynulé manuálne ovládanie drona počas letu.



Obr. 2: Zobrazenie viacerých dronov v aplikácii prenášanej do VR prostredia.

3.3 Pracovný priestor `ros2_ws`

Adresár `ros2_ws` predstavuje ROS2 pracovný priestor obsahujúci aplikáciu zodpovednú za autonómne riadenie misie, koordináciu roja dronov a komunikáciu s VR kontrolérmi.

Kľúčovou časťou systému je balík `swarm_mission`, ktorý zabezpečuje riadenie viacdronového scenára. Tento balík realizuje načítanie misijných bodov, koordináciu jednotlivých dronov a správu ich stavov počas letu. Každý dron komunikuje cez vlastný MAVROS namespace, čo umožňuje paralelnú správu viacerých zariadení v jednom systéme.

Koordinačný uzol zároveň implementuje mechanizmus prepínania medzi autonómnym a manuálnym režimom riadenia. Operátor môže počas misie prevziať kontrolu nad konkrétnym dronom, pričom systém zabezpečí korektné odovzdanie riadenia bez prerušenia ostatných autonómnych procesov. Po ukončení manuálneho zásahu môže byť dron opätovne zaradený do autonómneho režimu a pokračovať v plnení misie.

Súčasťou pracovného priestoru je aj modul zabezpečujúci prevod ROS2 kamera topikov zo simulácie na GStreamer video streamy. Tie sú následne prenášané do desktopovej aplikácie, kde ich operátor sleduje v reálnom čase. Architektúra systému je navrhnutá modulárne, čo umožňuje jednoduché rozšírenie o ďalšie drony, nové typy misií alebo doplnkové senzory.

4 Komunikácia medzi komponentmi

4.1 Dron a backendová aplikácia

Drony komunikujú s backendovou aplikáciou pomocou ROS topikov/servisov a v prípade reálneho dronu zdieľanie videa prebieha pomocou GStreameru.

Backend sa pripája na topiky:

- **/droneX/stop** - std_msgs/msg/Bool - požiadavka na zastavenie misie a pristátie po dokončení aktuálneho cyklu
- **/droneX/state** - mavros_msgs/msg/State - aktuálny stav dronu (stav pripojenia, mód letu, armed stav)
- **/droneX/local_position/pose** - geometry_msgs/msg/PoseStamped - aktuálna lokálna pozícia dronu
- **/droneX/image_raw** - sensor_msgs/msg/Image - obraz z kamery zo simulácie, konvertuje na GStreamer stream

Ďalej publikuje topiky, na ktoré sa dron pripája:

- **/droneX/setpoint_position/local** - geometry_msgs/msg/PoseStamped - publikovanie cieľovej pozície počas navigácie medzi waypointmi

A na ovládanie drona využíva servisy:

- **/droneX/set_mode** - mavros_msgs/srv/SetMode - prepnutie letového režimu dronu (napr. GUIDED)
- **/droneX/cmd/arming** - mavros_msgs/srv/CommandBool - arm/disarm dronu
- **/droneX/cmd/takeoff** - mavros_msgs/srv/CommandTOL - vzlet dronu do zadanej výšky
- **/droneX/cmd/land** - mavros_msgs/srv/CommandTOL - pristátie dronu

4.2 Backendová a frontendová aplikácia

Backendová a frontendová aplikácia spolu komunikujú cez rozhranie ROS2 bridge, ktoré umožňuje prenos správ a dát medzi ROS2 aplikáciami a aplikáciami nevyužívajúcimi ROS2. Tento spôsob sme zvolili z dôvodu vyššej portability aplikácie, bez nutnosti implementovať ROS2 klienta priamo vo VR aplikácii.

Front end sa pripája na topiky (pre každý dron zlášť):

- **/droneX/local_position/pose** - geometry_msgs/msg/PoseStamped - aktuálna pozícia robota

- **/droneX/battery** - sensor_msgs/msg/BatteryState - stav batérie robota
- **/droneX/state** - mavros_msgs/msg/State - aktuálny stav robota/dronu
- **/droneX/supervisor/handover_state** - std_msgs/msg/String - aktuálny stav handover

Front end publikuje riadiace topiky:

- **/supervisor/manual_cmd_vel** - geometry_msgs/msg/Twist - ovládanie smeru robota
- **/supervisor/takeover_request** - std_msgs/msg/String - prevzatie kontroly (jednorazový)
- **/supervisor/release_request** - std_msgs/msg/String - skončenie manuálneho režimu (jednorazový)

Video je streamované pomocou GStreamer-u.

4.3 Frontendová aplikácia a VR headset

Frontendová aplikácia využíva SteamVR na komunikáciu s VR headsetom, ktoré zabezpečuje prenos obrazu z počítača a OpenVR na čítanie vstupov z VR ovládačov, ktoré vieme následne v aplikácii spracovať a ovládať tak drona. Pre správne fungovanie Stremu, využívame open source projekt ALVR nainštalovaný ako server na PC a ako klient na VR headsete, keďže Pico Neo 3 nemá natívnu podporu SteamVR.

5 Misie a pracovný tok

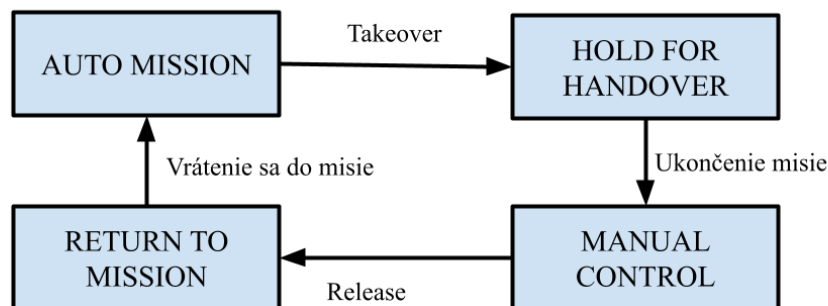
Misijné súbory v pracovnom priestore definujú postupnosť waypointov a špeciálnych príkazov. Každý dron má vlastný zoznam bodov, preto môže pokračovať v autonómnej misii nezávisle od ostatných dronov. Pri prevzatí jedného drona do manuálneho režimu ostatné drony nemenia svoj stav a pokračujú podľa svojich aktuálnych misijných stavov.

Podporovaný koncept misie zahŕňa:

- vzlet do pracovnej výšky,
- presun cez definované body,
- zmenu orientácie podľa špeciálnych príkazov,
- pristátie alebo kombinovaný príkaz pristátia a opätovného vzletu,
- uchovanie aktuálneho indexu misie pri dočasnom manuálnom zásahu.

6 Handover mechanizmus

Handover mechanizmus je kľúčovou časťou systému. Zabezpečuje, aby operátor mohol bezpečne prevziať manuálne riadenie jedného vybraného drona bez narušenia stability systému a ovplyvnenia ostatných autonómnych dronov.



Obr. 3: Zjednodušený stavový automat handover mechanizmu

V autonómnom režime dron sleduje waypointy z misijného súboru. Po prijatí požiadavky **takeover** sa vybraný dron presunie do stabilizačného stavu **HOLD_FOR_HANDOVER**. Po stabilizácii sa aktivuje **MANUAL_CONTROL**, v ktorom sú príkazy z operátora smerované iba na daný dron. Po požiadavke **release** sa rýchlostný príkaz vynuluje a dron sa vráti do autonómneho režimu.

7 Video streamovanie

Video streamovanie je oddelené od telemetrie. Obráz z dronov sa prenáša cez GStreamer, pričom stream zo simulácie sa z ROS2 topiku konvertuje na UDP H.264 stream. Desktopová aplikácia jednotlivé streamy prijíma, dekóduje a zobrazuje v paneloch dronov.

Príkladom mapovanie streamov je:

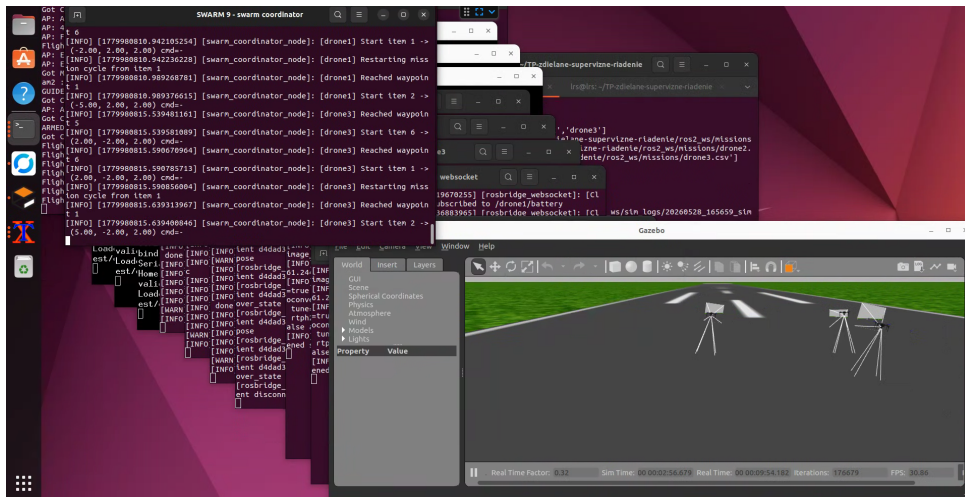
Dron	ROS2 topic	UDP port
drone1	/drone1/image_raw	5601
drone2	/drone2/image_raw	5602
drone3	/drone3/image_raw	5603

Tabuľka 1: Príklad mapovania video streamov

8 Testovacie spustenie

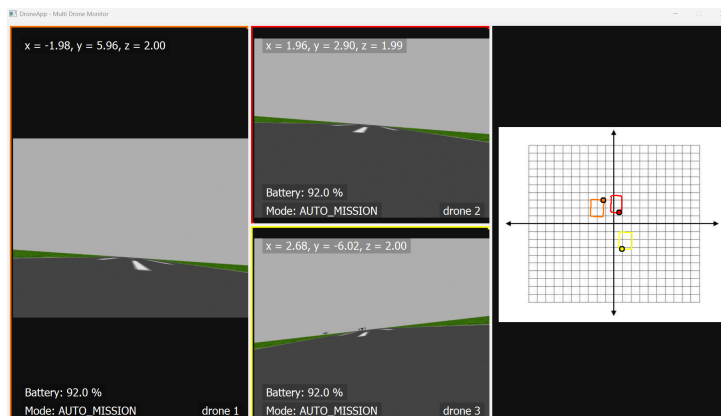
Na otestovanie fungovania aplikácie sme spustili aplikáciu podľa Používateľskej príručky (Príloha 1). Spustili sme simulačné prostredie na Linux počítači, frontend aplikáciu na Windows počítači a cez ALVR + SteamVR sme sa pripojili na VR headset.

Na obrázku 4 potom môžeme vidieť Gazebo simuláciu s simulačným svetom a terminály na ktorých beží simulácia a backendová aplikácia.

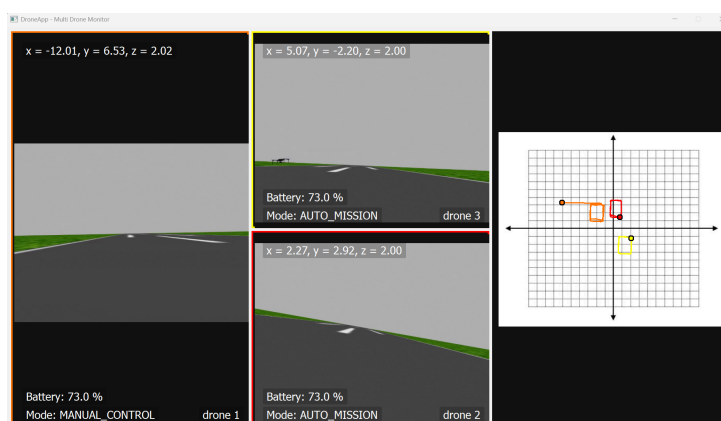


Obr. 4: Backend na Linux počítači.

Na obrázkoch 5 a 6 zase môžeme vidieť frontendovú aplikáciu bežiacu na Windows počítači v rôznych časových okamihoch.



Obr. 5: Frontendová aplikácia.



Obr. 6: Frontendová aplikácia po uplynutí istého času.

9 Záver

Spracovaný systém predstavuje modulárne riešenie pre zdieľané supervízne riadenie autonómnych dronov. ROS2 pracovný priestor zabezpečuje misiu, stavový automat a komunikáciu s MAVROS, zatiaľ čo DroneApp poskytuje operátorovi monitorovanie, výber aktívneho drona, video streamy a manuálne zásahy. Dokumentačná časť doc slúži ako návrhový a validačný podklad.

Najdôležitejším prvkom systému je riadený handover medzi autonómnym a manuálnym režimom. Vďaka nemu môže operátor zasiahnuť do letu jedného drona bez prerušenia činnosti celej skupiny. Takýto návrh je vhodný pre ďalší vývoj smerom k plnohodnotnému VR pracovisku pre dohľad nad autonómnymi robotickými systémami.